

To-Be-Fixed-2: Voice RAG Retrieval Issues

Sarkari Sahayak — Retrieval strategy review before voice pipeline integration

1. Whole-scheme documents are large for voice synthesis

Each retrieved document is a full scheme (document_text), sometimes 1000+ words. Feeding k=3 full documents into the answering LLM on every turn increases latency and token cost more than necessary for a voice agent, where response speed is felt directly in conversation. Consider returning only the most relevant section/field per scheme rather than the entire blob, or a lighter summarization step before the LLM call.

2. No conversation-context handling in search_schemes()

Voice conversations are multi-turn. A follow-up like "what documents do I need for that scheme" has no scheme name in the raw query text, so retrieval will likely miss or return unrelated results. The voice pipeline layer needs to either rewrite the query with context before calling search_schemes() (e.g. insert the last-mentioned scheme name), or pass the last-mentioned scheme_id through to filter/boost toward it.

3. Embedding call is a synchronous network round-trip per query

Cloudflare Workers AI embedding happens on every search_schemes() call, adding real latency inside the voice turn-taking budget on top of Weaviate + reranking (both fast/local). End-to-end latency for a realistic query should be measured before assuming this is acceptable in a live conversation — this is the piece most likely to introduce variable/slow response times.

4. No caching for repeated or common queries

Common questions (e.g. "what schemes for farmers") re-embed and re-search every time with no caching layer. Not urgent immediately, but worth addressing once real usage patterns are observed and repeat questions are common.